Chapter 17: Hash Tables and Collision Resolution

EECS 281 notes, condensed. Covers 17.1 to 17.7.

Legend

- **Red box** = **out of syllabus** (per your course notes). Skim only. These are:
  - 17.5.3 `std::unordered_multimap` / `std::unordered_multiset` (usage). *Takeaway kept in core:* map a key to a container instead.
  - 17.6.2 The seeded (Boost-style) hash combiner and the bit operators it uses
  - 17.6.3 Implementing a composite hash for a custom type (hasher functor + `operator==`)
  - 17.5.1 `std::piecewise_construct` (the textbook itself calls it out of scope)

  Note: your course outline calls these 17.7 / 17.8. In this textbook version (v0.3) they are 17.5.3 and 17.6.
- ⋆ = core, high-yield material worth memorizing.
- Boxes: **blue** = definition, **green** = worked example, **yellow** = remark / watch out. In tables: **green** = just inserted, *DEL* = deleted flag.

17.1 The Dictionary ADT

Dictionary: an ADT holding **key-value pairs**. Supports **insert** a pair and **look up** the value for a key. It is just an interface; implementations differ.

- **Unordered** dictionary → **hash table** (this chapter): ⋆ average Θ(1) lookup.
- **Sorted** dictionary → binary search tree (ch. 18): sorted access, but loses Θ(1) lookup.

17.2 Introduction to Hashing

17.2.1 Hash tables

Scanning a `vector<Item>` for a name is Θ(n). If keys were small non-negative ints, we could index an array directly in Θ(1). **Idea:** convert any key into an index, then index an array.

Hash table (hash map): dictionary implementation with **average-case Θ(1)** insert, lookup and removal. **Hashing** turns a key into an array index.

17.2.2 Hash and compression functions

Two steps: **key → hash function → integer → compression → index in [0, M)**, where M is the table size.

```
int32_t hash(const std::string& s) {
  if (s.empty()) return 0;
  return s[0] - 'A';   // 'A'→0, 'B'→1, ...
}
int32_t compress(int32_t h) {
  return h % table_size;   // h(t) mod M
}
```

M = 10:
`"Apple"` → hash 0 → index 0
`"Milk"` → hash 12 → 12 mod 10 → index 2
Lookup `"Apple"`: hash → go to index 0 → confirm key matches → return value. Θ(1).

If h(t) can be negative, take |h(t)| before the mod.

17.2.3 Properties of hash and compression functions

A valid hash function must…	Why
Be **easy to compute**	If it is slower than Θ(1), the table loses its point.
Compute a hash for **every** key	No key may crash or be unhashable.
Give the **same hash for the same key** (deterministic)	⋆ Otherwise you can never find the key again. (So `rand()` is a terrible hash.)
Should **distribute keys evenly**	Not required for validity, but needed for speed. Two keys at one index = **collision**.

Remark: "always return 0" is a *valid* hash function (meets the 3 musts) but a terrible one: every insert collides. Collisions can be reduced, never eliminated. A good hash does **not** keep similar keys close together.

Compression methods (map h(t) into [0, M)):

- **Division:** `|h(t)| mod M` ⋆ M should be **prime**.
- **MAD (Multiply-Add-Divide):** `|a·h(t) + b| mod M`, a and b prime, **a mod M ≠ 0**. Use when you can't choose M.

Why prime M? ⋆ Any key K sharing a factor t with M lands on an index that is a multiple of t (proof: $i = K - aM = t(u - av)$). E.g. M = 100 and all keys multiples of 5 → only indices 0, 5, 10, … used. M = 101 (prime) spreads them out.

Perfect hashing (no collisions): insert, search and remove are all Θ(1). This is a theoretical ideal, rarely reached in practice.

17.2.4 Addressing methods

- **Direct addressing:** keys come from a small universe {0, …, m−1} → array of size m, key = index. No hashing, no collisions.
- **Hashed addressing:** universe is huge (or much larger than the real key count) → hash + compress into a table of size m. Not one-to-one, so **collisions are inevitable**.

17.3 Collision Resolution Techniques

Example: `"Avocado"` hashes to 0, but `"Apple"` is already there. Four fixes: **separate chaining**, and three **open addressing** methods (**linear probing**, **quadratic probing**, **double hashing**).

17.3.1 Separate chaining

Separate chaining: each index holds a **linked list** of every key that hashed there. Collision → append to that list. Every key lives at the index it hashes to.

With N keys spread evenly over M slots, the average list length is **N/M** (≈ constant if M is big enough).

Operation	Average case (chaining)
Insert	Θ(1) if duplicates allowed; ⋆ Θ(N/M) if not (must scan list for the key first)
Search	Θ(N/M)
Erase	Θ(N/M)

17.3.2 Linear probing (open addressing)

Open addressing: one key per slot; on collision, look for **another empty slot** in the table.
Probe: one check of a slot. Outcomes: **Empty** (never used), **Deleted** (used before, now flagged), **Hit** (key matches), **Full** (occupied, different key).
Linear probing: probe `i, i+1, i+2, …` (mod M, wrapping around).

Insert "Avocado" (hash 0): probe 0 FULL, 1 FULL, 2 FULL, 3 EMPTY → placed at 3.

0	1	2	3	4	5	6	7	8	9
Apple	Banana	Milk	**Avocado**		Flour	Ginger			Tofu

Operation	Rule
Search	Probe from i until you hit the key (found) or an **Empty** slot (not in table). ⋆ **Deleted** slots count as occupied: keep probing.
Erase	Find the key, then replace it with a **DELETED flag** (a "tombstone"). Never just empty the slot.
Insert (no duplicates)	Probe like a search (Deleted = keep going) to confirm the key isn't already there. Stop at Empty. Then ⋆ insert at the **first Deleted or Empty** slot seen while probing.

Why the deleted flag? Say `"Jelly"` hashed to 9, collided with `"Tofu"`, and wrapped to 4. If erasing `"Tofu"` made slot 9 *Empty*, a search for `"Jelly"` would stop at 9 and wrongly report "not found." The flag says "something used to be here, keep probing."

Example 17.1 (M = 10, h(k) = k, c(k) = k mod 10, linear probing, no duplicates)
Ops: insert 13, 23, 33, 14 · erase 23 · insert 14 · erase 13 · insert 43

- 13→3; 23→3 taken→4; 33→3,4 taken→5; 14→4 taken,5 taken→6.
- Erase 23: probe 3 (13), then 4 → found → mark **DEL**.
- Insert 14: slot 4 is DEL, so keep probing → 14 found at 6 → **no insert**.
- Erase 13 → slot 3 = DEL.
- Insert 43: probe 3 (DEL), 4 (DEL), 5, 6, 7 (Empty) → not present → write at **first DEL/Empty = 3**.

Final table:

0	1	2	3	4	5	6	7	8	9
			43	*DEL*	33	14			

Example 17.2: how many probes before inserting 53 into this table? 53→3: probe 3, 4 (DEL), 5, 6, 7 (Empty) = **5 probes**; 53 goes to index **4** (first non-occupied slot seen).

17.3.3 Quadratic probing

Primary clustering: linear probing tends to fill runs of adjacent slots, and each run grows faster the bigger it is.
Half-full table, spread out: avg cost 0.5·1 + 0.5·2 = 1.5 = **Θ(1)**. Half-full, one big cluster: 0.5·1 + 0.5·(n/2) = **Θ(n)**.
Quadratic probing: probe `i, i+1², i+2², i+3², …` (mod M). Same insert/search/erase rules as linear; only the probe sequence changes.

Example 17.3 (M = 10, h(k) = k, quadratic) insert 13, 23, 33, 43, 53. All hash to 3.

- 13→3 · 23→3+1=4 · 33→3+1, 3+4=7 · 43→4, 7, 3+9=12→**2** · 53→4, 7, 2, 3+16=19→**9**

Final table:

0	1	2	3	4	5	6	7	8	9
		43	13	23			33		53

17.3.4 Double hashing

Secondary clustering: with quadratic probing, keys that hash to the same slot still follow the **same** probe sequence (63, 73, 83 all probe 3, 4, 7, 9, …).
Double hashing: use a second hash t′(key) to set the step size per key:

```
index = [ t(key) + j × t′(key) ] mod M
```

(j = collision number 0, 1, 2, …)

Different keys get different step sizes, so their probe sequences differ. Same insert/search/erase rules.

Example 17.4 (M = 10, t′(key) = 7 − (key mod 7)) insert 13, 23, 33, 25

- 13→3.
- 23→3 taken → 23 + 1·(7 − 2) = 28 → **8**.
- 33→3 taken → 33 + 1·(7 − 5) = 35 → **5**.
- 25→5 taken → 25 + 1·(7 − 4) = 28 → 8 taken → 25 + 2·3 = 31 → **1**.

Final table:

0	1	2	3	4	5	6	7	8	9
	25		13		33			23	

Method	Probe sequence after collision at i	Fixes	Weakness
Separate chaining	none, append to list at i	—	Extra memory for lists
Linear probing	i+1, i+2, i+3, …	—	Primary clustering
Quadratic probing	i+1², i+2², i+3², …	Primary clustering	Secondary clustering; can fail if α > ½
Double hashing	i+1·t′, i+2·t′, i+3·t′, …	Secondary clustering	Cost of a second hash

17.4 Load Factor and Dynamic Hashing

17.4.1 Load factor

Load factor: $\alpha = N / M$ (N = keys stored, M = table size). ⋆ **Deleted flags do not count** toward N.

- Chaining: α = average list length; can exceed 1.
- Open addressing: α = fraction of slots filled; **α ≤ 1**.

Knuth (linear probing, even hashing): expected probes for a successful search = $\frac{1}{2}(1 + 1/(1-\alpha))$; for an unsuccessful search or insert = $\frac{1}{2}(1 + 1/(1-\alpha)^2)$.

α	0.1	0.3	0.5	0.6	0.7	0.8	0.9
Successful	1.1	1.2	1.5	1.8	2.2	3.0	5.5
Unsuccessful	1.1	1.5	2.5	3.6	6.1	13.0	50.5

⋆ Rule of thumb: keep a linear-probing table **at most half full**.

Quadratic probing danger: (c + j²) mod M only produces about **half** of the possible remainders (quadratic residues). M = 7, h(k) = k, insert 9, 16, 20, 23 → slots 2, 3, 6, 4. Now 30 hashes to 2 and probes only 2, 3, 6, 4, 4, 6, 3, 2, 3, … **forever**, while slots 0, 1, 5 are free. So with quadratic probing, never let α go above ½.

Chaining vs open addressing as α grows: chaining degrades gradually (double N → lists double). Open addressing degrades sharply and is unusable at α = 1.

17.4.2 Dynamic hashing (resizing)

Dynamic hashing: when α exceeds a threshold (commonly **0.5**), **double M** and **rehash every element** into the new table. Deleted slots are not rehashed.

Why rehash, not copy? An index depends on M. Key 37: 37 mod 4 = **1** in the old table, 37 mod 8 = **5** in the new one. Copying slot-for-slot would make keys unfindable.

Amortized cost: resizes happen on inserts 2, 3, 5, 9, … (cost 1 + old size). T(n) = n (inserts) + (1 + 2 + 4 + … + n′) (rehashing) ≤ n + (2n − 1) = Θ(n). So ⋆ **amortized insert = Θ(n)/n = Θ(1)**, even though a single insert can be Θ(n).

17.5 STL Unordered Map and Set

17.5.1 std::unordered_map<K, V> (`<unordered_map>`)

Stores key-value pairs with **unique keys**, in no particular order. Typically a hash table with **separate chaining**. Keys must be hashable (`int`, `double`, `std::string` work out of the box).

```
std::unordered_map<std::string, double> food_prices;
food_prices.insert({"Apple", 3.99});
food_prices["Tofu"] = 3.79;                  // operator[] also inserts
std::cout << food_prices["Tofu"];           // 3.79
```

Member	Behavior
`insert({k, v})`	Inserts only if k is absent. Returns `pair<iterator, bool>` (iterator to the element, and whether it was inserted). ⋆ Does **not** overwrite an existing value.
`emplace(args...)`	Like insert, but builds the pair in place from constructor args: `emplace("Banana", 2.49)`.
`try_emplace(k, args...)` (C++17)	Like emplace, but if k already exists it **constructs nothing** and doesn't move from args. Best for expensive or move-only values. Args build the value directly.
`operator[](k)`	Returns `V&`. ⋆ **If k is absent, inserts it** with a value-initialized value (0, "", …).
`find(k)`	Iterator to the element, or `end()` if absent. ⋆ Preferred safe lookup.
`count(k)`	0 or 1 (keys are unique).
`erase(k)` / `erase(it)` / `erase(first, last)`	By key returns count removed (0 or 1); by iterator returns iterator to the next element.
`size()`, `empty()`, `clear()`	As usual.
`begin()`/`end()`	Iterate over `pair<const K, V>` (`it->first` = key, `it->second` = value). Every element is visited, but the **order is unspecified**.

Complexity (map & set)	Insert	Access	Erase	Find
Average	Θ(1)	Θ(1)	Θ(1)	Θ(1)
Worst	Θ(n)	Θ(n)	Θ(n)	Θ(n)

Watch out, `operator[]` creates keys: `days_in_month[input]` with a bad input like `"Feb"` silently inserts `{"Feb", 0}`. Use `find()` when the key might not exist. Use `[]` only when you know it's there, or when auto-insert is what you want (e.g. `++word_count[word]`).

Avoid repeated lookups (each one hashes the key):

```
// BAD: 2 lookups
if (m.find(key) != m.end())
  cout << m[key];
// BAD: 3 lookups
emp[name].title = t;
emp[name].salary = s;
emp[name].years = y;
```

```
// GOOD: 1 lookup
auto it = m.find(key);
if (it != m.end())
  cout << it->second;
// GOOD: 1 lookup (note the &)
auto& e = emp[name];
e.title = t; e.salary = s; e.years = y;
```

Prefer `find()` over `count()` + `[]` for the same reason. Insert returns a pair, so structured bindings are handy: `auto [it, ok] = m.try_emplace("Apple", 3.99);`

❄ OUT OF SYLLABUS · std::piecewise_construct

`emplace` still may copy the key/value while building the pair. `std::piecewise_construct` avoids this by passing separate constructor-arg tuples:

```
course_map.emplace(std::piecewise_construct,
                   std::forward_as_tuple("EECS", 281),
                   std::forward_as_tuple(12345678, "Bob", "Smith"));
```

The textbook itself calls this out of scope. `try_emplace` is the simpler C++17 alternative.

Example 17.5: word count

```
void print_word_count(std::istream& words) {
  std::unordered_map<std::string, int32_t> counts;
  std::string w;
  while (words >> w) ++counts[w];          // [] auto-inserts 0: intended here
  for (const auto& [word, n] : counts) std::cout << word << " " << n << '\n';
}
```

Output order is arbitrary. Use `std::map` (ch. 18) if you need sorted order.

Remark: don't overuse hash tables. Small integer keys → just use a `vector` (direct addressing). Hash tables cost more memory and hashing work.

17.5.2 std::unordered_set<K> (`<unordered_set>`)

Unique elements, no order; the value **is** the key. Use it for "have I seen this?" Same members and complexities as the map, **except no `operator[]`**. Extra insert overloads: `insert(first, last)` and `insert({1, 3, 5})`.

Example 17.6: first repeated element, Θ(n)

```
int32_t first_repeated(const std::vector<int32_t>& vec) {
  std::unordered_set<int32_t> seen;
  for (int32_t v : vec) {
    if (seen.find(v) != seen.end()) return v;
    seen.insert(v);
  }
  return -1;
}
```

17.5.3 Mapping one key to many values

⋆ **Takeaway (in syllabus):** to map a key to **multiple values**, use a regular `unordered_map` whose value is a **container**, such as `std::unordered_map<K, std::vector<V>>`, instead of a multimap.

```
std::unordered_map<std::string, std::vector<int32_t>> classes;
classes["EECS"].push_back(280);
classes["EECS"].push_back(281);
classes["MATH"].push_back(217);
auto it = classes.find("EECS");
if (it != classes.end())
  for (int32_t num : it->second) std::cout << num << '\n';
```

❄ OUT OF SYLLABUS · 17.5.3 unordered_multimap / unordered_multiset

- `std::unordered_multimap<K, V>`: an unordered map that allows **duplicate keys**. `std::unordered_multiset<K>`: a set that allows duplicates.
- Get every element with a key via `equal_range(k)`, which returns a `pair<iterator, iterator>` range [first, second). Order within the range is unspecified.

```
auto [lo, hi] = classes.equal_range("EECS");
for (auto it = lo; it != hi; ++it) std::cout << it->second << '\n';
```

17.6 STL Hashing and Composite Hash Functions

17.6.1 std::hash<> (`<functional>`)

`std::hash<T>` is a **function object (functor)** that returns a `size_t` hash value using the standard library's default hash for T (works for `int`, `double`, `std::string`, ...).

```
std::hash<std::string> hasher;      // create the functor
size_t h = hasher("EECS281");       // call it like a function
```

The exact function depends on the implementation, but it's deterministic and collisions between different keys are very unlikely.

17.6.2 Composite hash functions

To hash an object with several parts (e.g. `struct Coordinate { int32_t x, y; };`), combine the hashes of its components. A **composite hash**:

$$H(\{x, y\}) = h(x) + p \times h(y)$$

e.g. h(15) = 10, h(−27) = 20, p = 5 → H = 10 + 5·20 = 110.

⋆ **Know this: linearly combining** component hashes is **not** a good way to hash an object with many parts, because collisions become easy to predict.

❄ OUT OF SYLLABUS · 17.6.2 The seeded hash combiner (Boost hash_combine)

For each component k_i: `seed ^= h(k_i) + 0x9e3779b9 + (seed << 6) + (seed >> 2)`, starting from seed = 0.

```
void hash_combine(size_t& seed, int32_t v) {
  std::hash<int32_t> hasher;
  seed ^= hasher(v) + 0x9e3779b9 + (seed << 6) + (seed >> 2);
}
```

Bit operators (EECS 370 material): `^=` is XOR-assign (bit = 1 where the bits differ, e.g. 281 ^ 370 = 107); `<< n` shifts left n bits (×2 each); `>> n` shifts right n bits (÷2 each, floored).

❄ OUT OF SYLLABUS · 17.6.3 Hashing a custom type

To use a custom key in `unordered_map`/`unordered_set` you need (1) a **hasher functor** with `size_t operator()(const T&) const` and (2) **equality**, usually by overloading `operator==`. Pass the hasher as a template argument.

```
struct Coordinate {
  int32_t x, y;
  bool operator==(const Coordinate& o) const { return x == o.x && y == o.y; }
};
struct CoordinateHasher {
  size_t operator()(const Coordinate& c) const {
    size_t seed = 0;  /* hash_combine(seed, c.x); hash_combine(seed, c.y); */  return seed;
  }
};
std::unordered_map<Coordinate, std::string, CoordinateHasher> location_map;   // 3rd arg
std::unordered_set<Coordinate, CoordinateHasher> location_set;                 // 2nd arg
```

17.7 Solving Problems Using Hash Tables

Pattern: ⋆ if an inner loop exists only to *look something up*, replace it with a hash table: Θ(n²) → Θ(n) average, at the cost of Θ(n) extra space.

Example 17.7: two-sum on an unsorted array (two pointers needs sorted input). For each value, check whether it's a needed complement; otherwise store `target − val`. Θ(n) time, Θ(n) space.

```
std::pair<int32_t,int32_t> target_sum(const std::vector<int32_t>& vec, int32_t target) {
  std::unordered_set<int32_t> complements;
  for (int32_t val : vec) {
    if (complements.find(val) != complements.end()) return {val, target - val};
    complements.insert(target - val);
  }
  return {-1, -1};
}
```

[7, 10, 3, 9, 4], target 11: set grows {4}, {4, 1}, {4, 1, 8}, {4, 1, 8, 2}; then 4 is in the set → return [4, 7].

Example 17.8: office-hours rate limit (allow a join once per 60 min). Map name → last join time. Θ(1) average per call.

```
std::unordered_map<std::string, int32_t> oh_map;
bool allow_to_join(const std::string& name, int32_t t) {
  auto it = oh_map.find(name);
  if (it == oh_map.end()) { oh_map[name] = t; return true; }   // first time
  if (t - it->second >= 60) { it->second = t; return true; }  // cooldown done
  return false;                                                // too soon
}
```

Example 17.9: LRU cache (get/set in Θ(1), evict the least-recently-used item at capacity)

- Timestamps → Θ(n) scan to find the LRU. Priority queue → Θ(log n). **Doubly-linked list** → Θ(1).
- **`std::list`** keeps items in recency order: front = most recent, back = LRU. Move to front on every access, evict from the back.
- **`unordered_map<key, list::iterator>`** finds a key's node in Θ(1). List iterators stay valid until that node is erased.

```
std::list<std::pair<std::string, double>> cache_list;
std::unordered_map<std::string, decltype(cache_list)::iterator> cache_map;

std::optional<double> get(const std::string& key) {
  auto it = cache_map.find(key);
  if (it == cache_map.end()) return std::nullopt;
  cache_list.splice(cache_list.begin(), cache_list, it->second);  // move node to front
  return it->second->second;
}
void set(const std::string& key, double value) {
  auto it = cache_map.find(key);
  if (it != cache_map.end()) {
    cache_list.splice(cache_list.begin(), cache_list, it->second);
    it->second->second = value;
    return;
  }
  if (cache_map.size() == capacity) {                // evict LRU (back)
    cache_map.erase(cache_list.back().first);
    cache_list.pop_back();
  }
  cache_list.emplace_front(key, value);
  cache_map[key] = cache_list.begin();
}
```

Example 17.10: common strings with the minimum index sum

- Naïve nested loops: Θ(mn).
- Better: put one vector into `unordered_map<string, size_t>` (string → index). Scan the other vector, look each string up in Θ(1), and track the best sum (clear the result on a new best, append on a tie). **Θ(m + n)**.
- Optimization: put the **smaller** vector into the map to save memory.

Quick Review

Fact	Answer
Hash table op cost	Average Θ(1); worst Θ(n) (everything collides)
Three "musts" of a hash function	Fast to compute, defined for every key, deterministic. ("Should": spread keys evenly)
Best table size for division compression	A prime
MAD condition	|a·h + b| mod M with a mod M ≠ 0
Search stops at…	The key (hit) or an **Empty** slot. Deleted slots mean keep going.
Insert goes to…	The first Deleted *or* Empty slot seen, after confirming the key is absent
Load factor	α = N/M (deleted slots excluded); resize (double M + rehash) when α > 0.5
Amortized insert with doubling	Θ(1)
STL unordered containers use…	Separate chaining (typically)
`m[k]` on a missing key	Inserts k with a value-initialized value
One key → many values	`unordered_map<K, vector<V>>`
Hashing a multi-part object	Combine component hashes (`std::hash`); a plain linear combination is a weak choice